

CS 3510 Algorithms: Finding Minimum Spanning Trees with Greedy Algorithms

Aaron Hillegass
Georgia Tech

Minimum Spanning Subgraph?

You have a connected undirected weighted graph G with a finite number of vertices.

You want to find a connected subgraph T that contains all vertices of G and has minimum total edge weight.

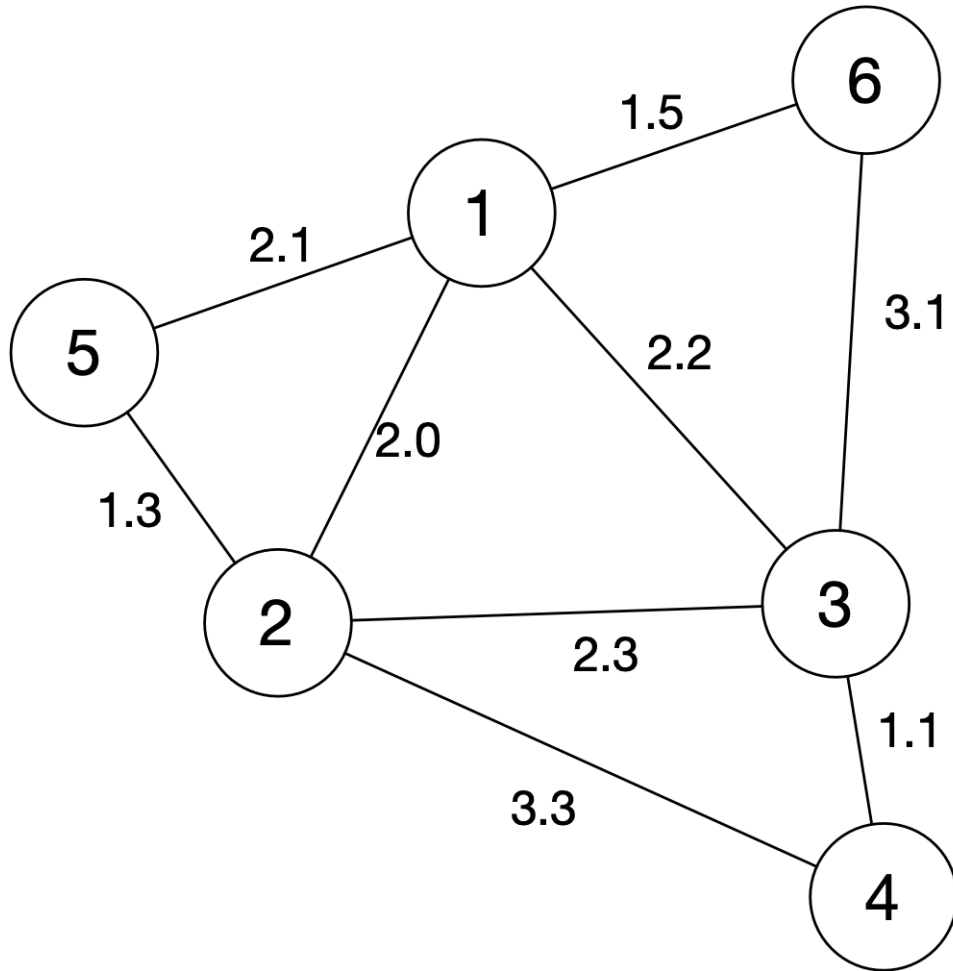
Given a connected undirected graph G , the following are equivalent:

1. G is a tree
2. G has $|V| - 1$ edges
3. G has no cycles

Why is the minimum spanning subgraph always a tree?

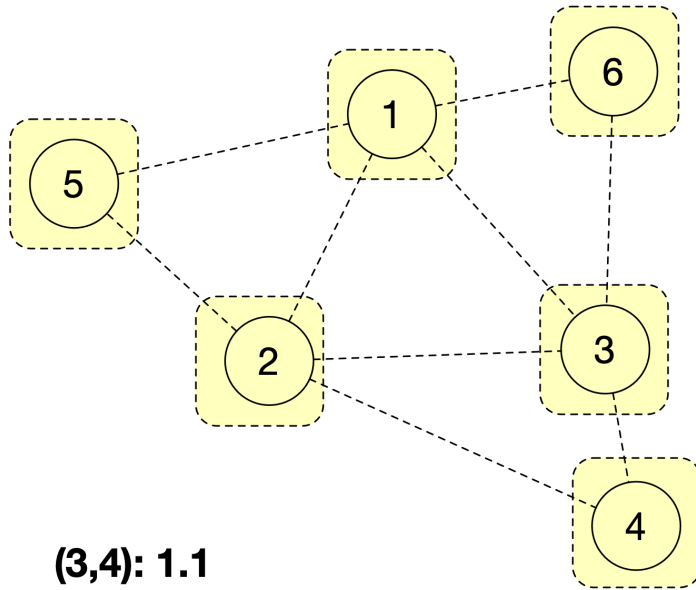
Example

4/26

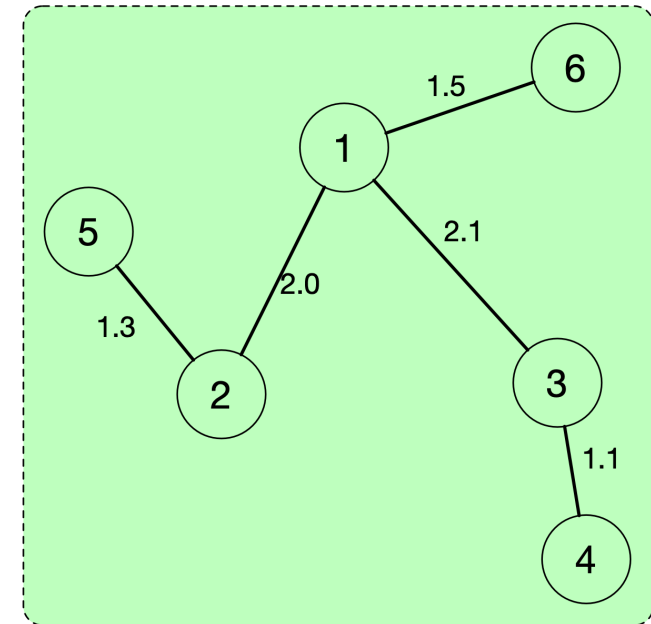
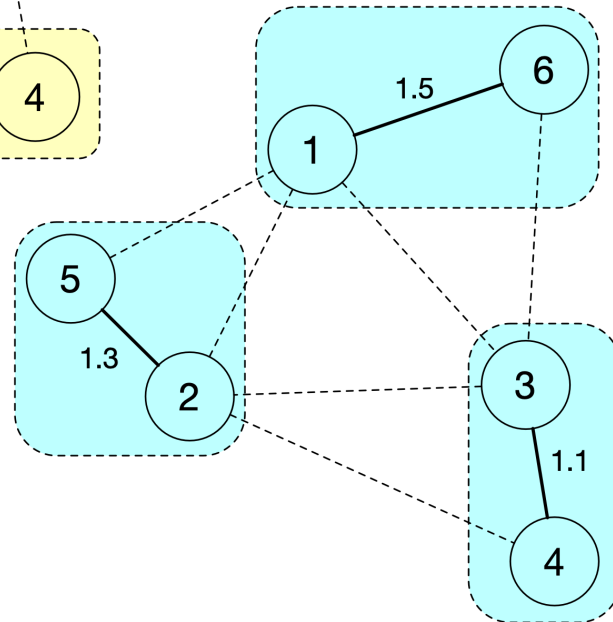


(3,4): 1.1
(2,5): 1.3
(1,6): 1.5
(1,2): 2.0
(1,5): 2.1
(1,3): 2.2
(2,3): 2.3
(3,6): 3.1
(2,4): 3.3

Example



(3,4): 1.1
(2,5): 1.3
(1,6): 1.5
(1,2): 2.0
 (1,5): 2.1
(1,3): 2.2
(2,3): 2.3
 (3,6): 3.1
 (2,4): 3.3



Kruskal's Algorithm 1

```
1 define kruskals( $G$ ):  
2   edges := sorted_by_cost( $G.E$ )  
3   forest :=  $G.V$ , no edges  
4   while len(forest) > 1:  
5     ( $u, v$ ) := pop_head(edges)  
6     if  $u$  and  $v$  are in different trees:  
7       | create an edge between  $u$  and  $v$   
8   return forest[1]
```

Sort? $O(|E| \log |E|)$

Loop? $O(|E|) \times \text{check} + O(|V|) \times \text{merge}$

Three functions:

- `create_djset( $n$ )`: Creates a disjoint set with integers $1..n$, each as its own set
- `find( $x$ )`: Finds the representative of the set containing x , such that $\text{find}(x) = \text{find}(y)$ if and only if x and y are in the same set.
- `union( $x, y$ )`: x and y are representatives of two sets. This merges them into one set.

```
s := create_djset(5)
set_a := s.find(3)
set_b := s.find(4)
if set_a != set_b:
    s.union(set_a, set_b)
```

Kruskal's Algorithm 2

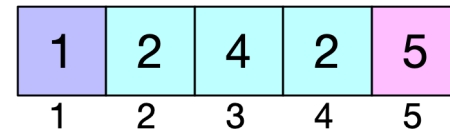
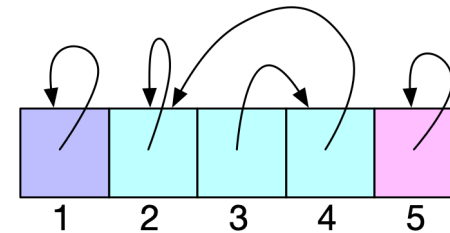
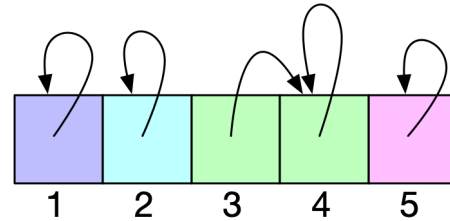
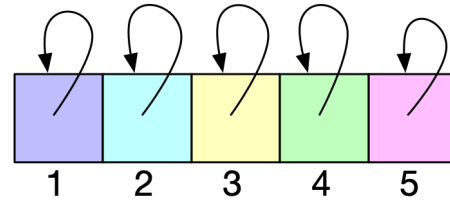
```
1 define kruskals( $G$ ):
2   edges := sorted_by_cost( $G.E$ )
3   trees := create_djset( $|G.V|$ )
4   tree_edges := []
5   while trees.count > 1:
6     ( $u, v$ ) := pop_head(edges)
7     u_set := trees.find( $u$ )
8     v_set := trees.find( $v$ )
9     if u_set does not equal v_set:
10      trees.union(u_set, v_set)
11      tree_edges.append( $(u, v)$ )
12  return tree_edges
```


Disjoint Set

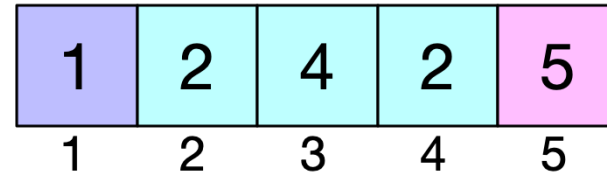
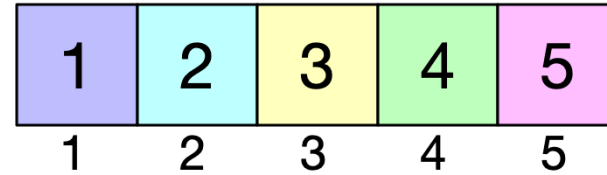
9/26

```
s := create_djset(5)
set_a := s.find(3)
set_b := s.find(4)
if set_a != set_b:
    s.union(set_a, set_b)

s.union(s.find(3), s.find(2))
```



```
1 define create_djset( $n$ ):  
2    $\text{parents} := \text{new array}(n)$   
3   forall  $i$  in  $1..n$ :  
4      $\text{parents}[i] := i$   
5    $\text{count} := n$   
6   return ( $\text{parents}, \text{count}$ )
```



```
1 define find( $s, x$ ):  
2   while  $x$  does not equal  $s.\text{parents}[x]$ :  
3      $x := s.\text{parents}[x]$   
4   return  $x$ 
```

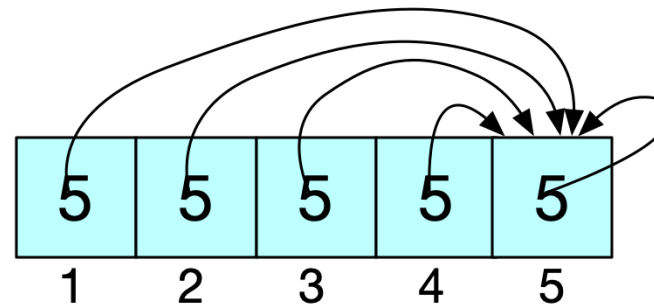
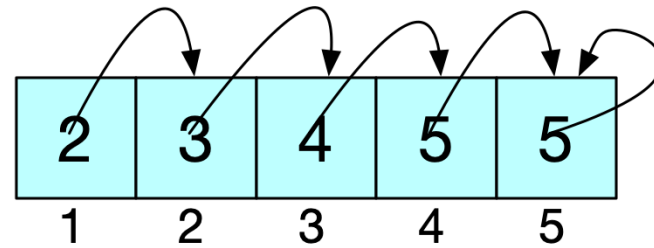
```
1 define union( $s, x\_root, y\_root$ ):  
2    $s.\text{parents}[x\_root] := y\_root$   
3    $s.\text{count} := s.\text{count} - 1$ 
```

Path Compression

A bushy tree is so much better than a linked list...

```
1 define find( $s, x$ ):  
2   parent :=  $s.parents[x]$   
3   if  $x = \text{parent}$ , return  $x$   
4   root := find( $s, \text{parent}$ )  
5    $s.parents[x] := \text{root}$   
6   return root
```

find(1)



How fast is a disjoint set?

With a little bookkeeping to guarantee the tree is bushy...

Insert: $O(1)$

Union: $O(1)$

Find: $O(\alpha(n)) < O(\log \log n)$ (amortized)

Complexity of Kruskal's

```
1 define kruskals( $G$ ):  
2   edges := sorted_by_cost( $G.E$ )  
3   trees := create_djset( $|G.V|$ )  
4   tree_edges := []  
5   while trees.count > 1:  
6     ( $u, v$ ) := pop_head(edges)  
7     u_set := trees.find( $u$ )  
8     v_set := trees.find( $v$ )  
9     if u_set does not equal v_set:  
10      trees.union(u_set, v_set)  
11      tree_edges.append( $(u, v)$ )  
12 return tree_edges
```

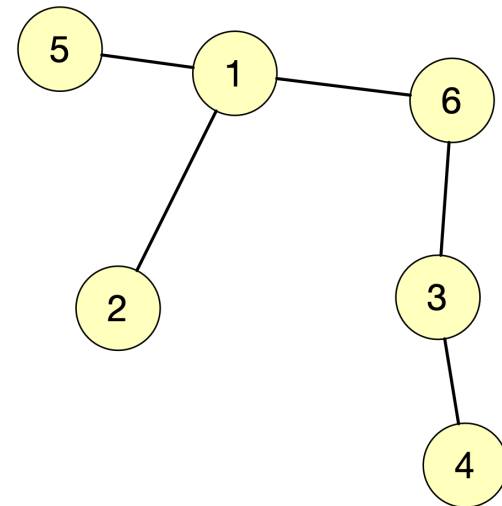
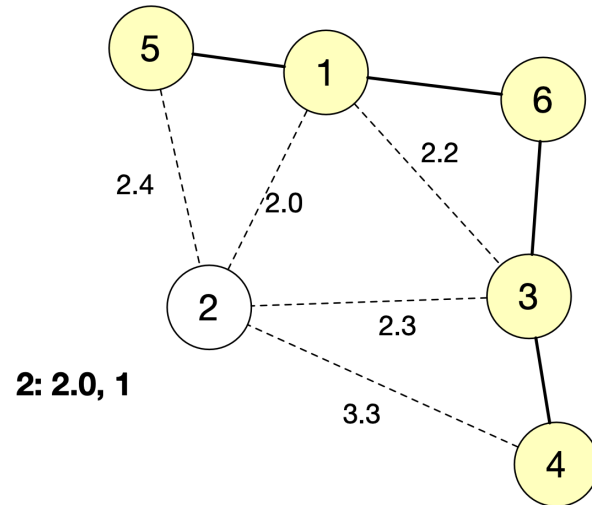
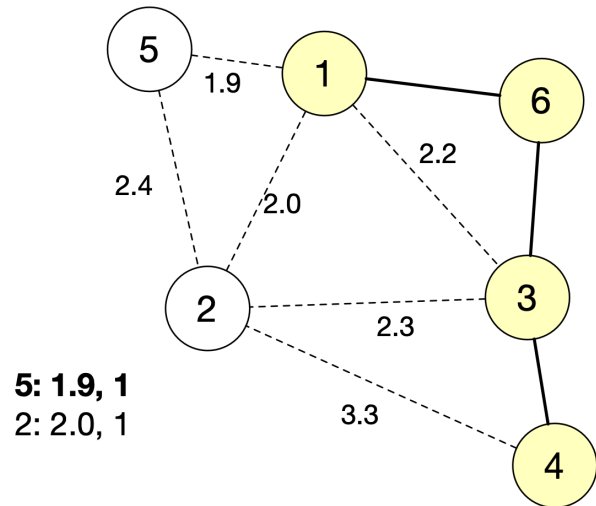
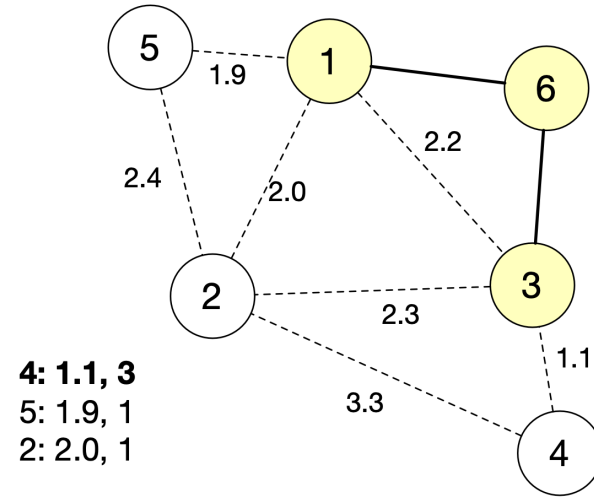
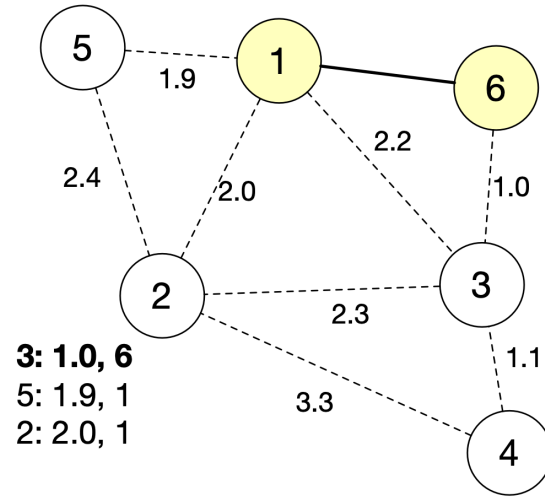
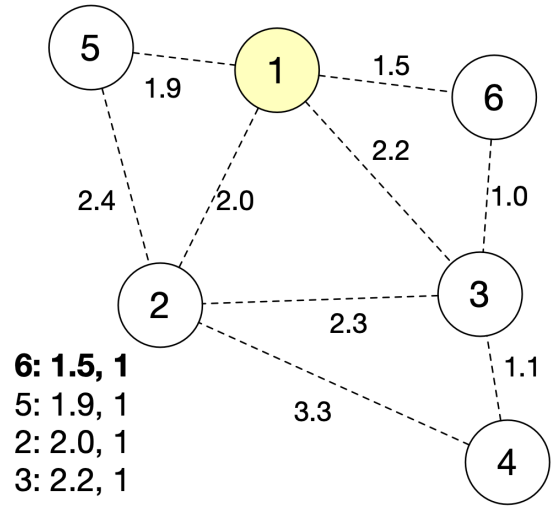
$$O(|E| \log |E|)$$

Is there another way to find the
minimum spanning tree?

Kruskal: "Make each vertex a tree. Merge trees that are close to each other until there is one tree." Uses Disjoint Set.

Prim: "Start with a single vertex as a tree. Add closest vertex to the tree until all vertices are in the tree." Uses Priority Queue.

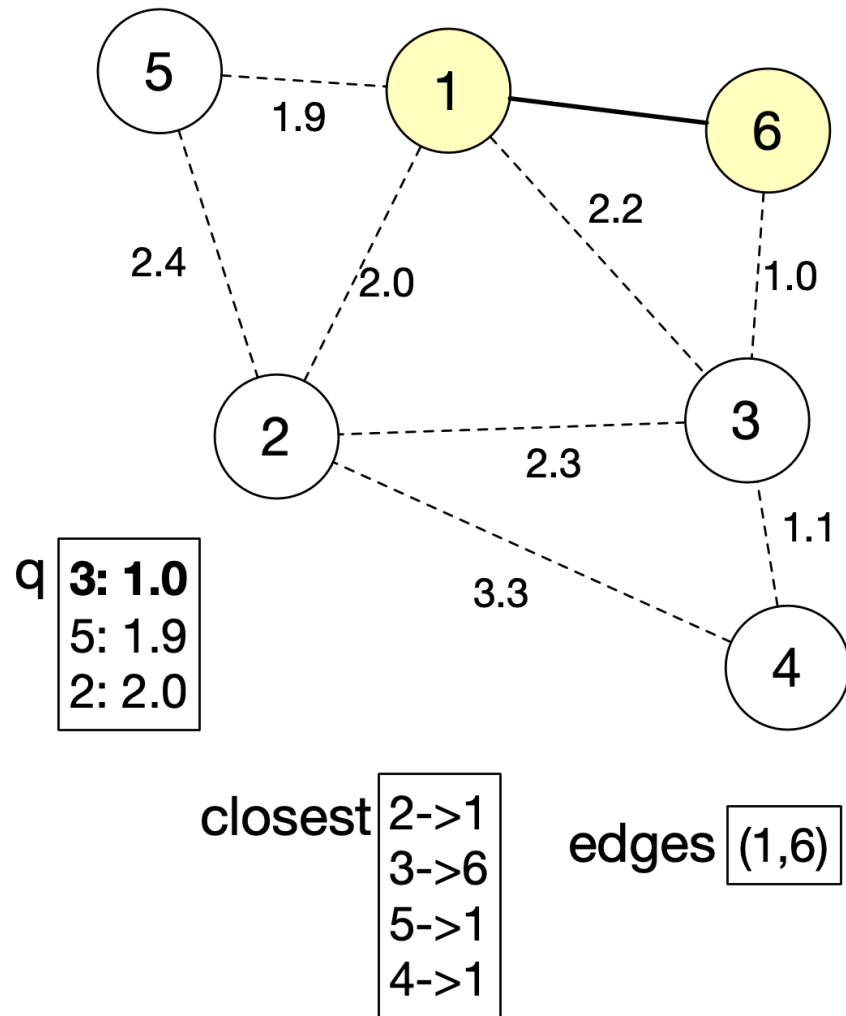
Prim's Example



Prim's Algorithm

17/26

```
1 define prims( $G$ ):  
2    $v :=$  some vertex in  $G.V$   
3   edges := [],  $q :=$  priority queue  
4   closest := new map  
5   for  $u$  in ( $G.V - v$ ), closest[ $u$ ] :=  $v$   
6   while closest.count > 0:  
7     for ( $\{v, \text{child}\}, \text{cost}$ ) in  $G.E$ :  
8       if child  $\in$  closest and  $q.\text{decrease}(\text{child}, \text{cost})$ :  
9         | closest[child] :=  $v$   
10     $v := q.\text{pop\_min}()$   
11    edges.append( $\{\text{closest}[v], v\}$ )  
12    delete  $v$  from closest  
13  return edges
```



Complexity of Prim's

```
1 define prims( $G$ ):  
2    $v :=$  some vertex in  $G.V$   
3   edges := [],  $q :=$  priority queue  
4   closest := new map  
5   for  $u$  in ( $G.V - v$ ), closest[ $u$ ] :=  $v$   
6   while closest.count > 0:  
7     for ( $\{v, \text{child}\}, \text{cost}$ ) in  $G.E$ :  
8       if child  $\in$  closest and  $q.\text{decrease}(\text{child}, \text{cost})$ :  
9         | closest[child] :=  $v$   
10       $v := q.\text{pop\_min}()$   
11      edges.append( $\{\text{closest}[v], v\}$ )  
12      delete  $v$  from closest  
13 return edges
```

$$O(|E| \log |V|)$$

Imagine G is not connected.

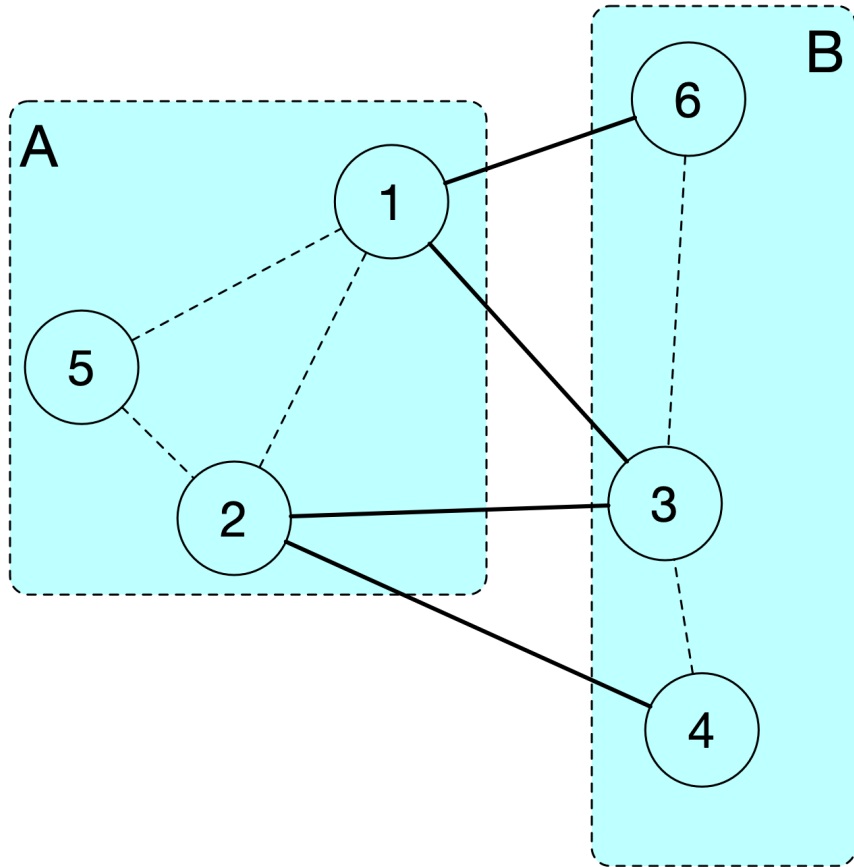
What happens when you run Prim's?

Kruskal's?

Always be suspicious of greedy algorithms!

- Most greedy algorithms always get you a solution.
- Few greedy algorithms are guaranteed to get you an optimal solution!
- But I claim Kruskal's and Prim's will.

Better check it.



Given an undirected graph $G = (E, V)$.

A *cut* is a partition of V into two non-empty disjoint sets A and B such that $A \cup B = V$ and $A \cap B = \emptyset$.

The *cut-set* of A and B is defined as the set of edges that have one endpoint in A and the other in B .

Given a spanning tree T and a cut, at least one edge of T is in the cut-set.

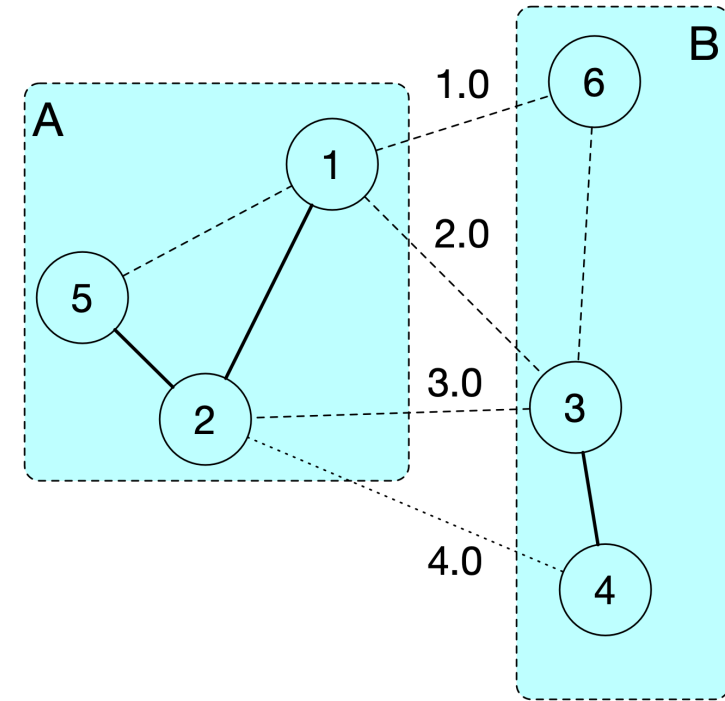
The Cut Property

Let X be some edges from a minimum spanning tree of G .

Let (A, B) be a cut such that no edges of X are in the cut-set.

Let e be the cheapest edge in the cut-set.

Then $X \cup \{e\}$ is part of some minimum spanning tree of G .



Cut Property Proof

Let $X \subset T$, where T is a MST of G .

If e is in T , done. So assume e is not in T .

Add e to T – introduces a cycle across the cut. Call the other edge involved e'

Create T' by adding e to and removing e' from T .

Removing an edge from a cycle doesn't disconnect a graph. So T' is a tree and has the same number of edges as T , so it is a spanning tree.

Is it minimal? $w(T') = w(T) - w(e') + w(e) \leq w(T)$. T was minimal, so T' must be.

Prim's starts with a single vertex v_0 and adds new vertices $v_1, v_2, \dots, v_n$.

Inductive proof: The single vertex with no edges is obviously a subset of an MST.

At each step i , the cut is $A = \{v_0, \dots, v_i\}$ and $B = V - A$.

If we assume the edges gathered are part of an MST, when we add the cheapest edge e in the cut, the result is part of an MST.

When all the vertices are in A , the edges are spanning. Thus we have an MST.

Kruskal's starts with the empty set and adds the shortest edge that connects two trees.

Inductive proof: The empty set is obviously a subset of an MST.

Every step in Kruskal's adds an edge that connects two trees, T_1 and T_2 .

Assume that before the step the edges gathered are part of an MST.

The edge we add is the cheapest edge in the cut-set for $(T_1, V - T_1)$.

Thus the new set of edges is also part of an MST.

We continue until all the vertices are in a single tree. Thus it spanning, thus an MST.

Questions?

Slides by Aaron Hillegass